

LEMBAR KERJA

PEMROGRAMAN INTERNET

Dasar CSS



DISUSUN OLEH:

Tim Asisten Praktikum Pemrograman Internet
2026/2027

LEMBAR KERJA

Pertemuan 3 | CSS Dasar: Sintaks, Selector, Box Model, dan Layout

Identitas Praktikan

Nama Lengkap	muhammad gabriel venusha	NIM	2511064
Kelas	PILKOM C	Tanggal	14 september 2026

Capaian Praktikum

Setelah menyelesaikan lembar kerja ini, praktikan diharapkan mampu:

- Menuliskan aturan CSS yang benar serta memilih cara penulisan inline, internal, atau eksternal sesuai kebutuhan.
- Menargetkan elemen dengan selector yang tepat dan menghitung spesifisitas ketika terjadi konflik aturan.
- Menerapkan box model dan unit ukuran yang sesuai untuk mengatur ukuran serta jarak antar elemen.
- Mengatur tipografi, warna, dan background sebuah halaman.
- Menyusun tata letak memakai display, position, Flexbox, dan CSS Grid.
- Membuat tampilan responsif dengan media query berdasarkan pendekatan mobile-first.

Petunjuk Pengerjaan

- Kerjakan seluruh bagian secara berurutan pada kolom jawaban yang tersedia.
- Jawaban teori ditulis dengan kalimat sendiri, bukan salinan modul.
- Jawaban berupa kode ditulis dengan indentasi yang konsisten karena kerapian ikut dinilai.
- Simpan berkas dengan format NIM_Nama_LK3.pdf dan unggah sesuai batas waktu yang diumumkan asisten.
- Unggah file yang berhubungan dengan tugas seperti file gambar,



kode program, dan sebagainya dalam bentuk .ZIP



A. Pemahaman Konsep

Jawab dengan kalimat sendiri, cukup dua sampai empat kalimat per nomor.

A.1. Sebelum CSS ada, styling ditulis langsung di dalam HTML memakai tag dan atribut presentasi. Jelaskan dua masalah yang muncul dari cara tersebut dan bagaimana CSS mengatasinya.

Jawaban:

styling ditulis berulang di tiap tag (kyk `<font color="blue">` di banyak tempat), jadi kalau mau ganti warna harus edit satu-satu, mengatasinya cukup buat satu aturan pakai selector, ke semua elemen sekaligus.

HTML jadi berat dan berantakan. Struktur konten bercampur dengan detail tampilan, file jadi besar dan susah dibaca. mengatasinya, styling dipindah ke file terpisah (.css), jadi si htmlny jadi bersih dan bisa di-cache browser (lebih cepat).

A.2. Sebutkan tiga cara menghubungkan CSS ke dokumen HTML, lalu jelaskan mengapa cara eksternal paling direkomendasikan untuk proyek yang terdiri atas banyak halaman.

Jawaban:

1. inline css dengan ditulis lngsung di atribut style pada tag
2. Internal CSS — ditulis di dalam tag `<style>` pada bagian `<head>` dokumen.
3. External CSS — ditulis di file .css terpisah, lalu dihubungkan pakai tag `<link>`.

karna satu file, dipakai bersama — semua halaman bisa "nyambung" ke file CSS yang sama, jadi cukup kelola satu tempat aja.

Konsisten — tampilan otomatis seragam di semua halaman karena sumber stylenya sama.

Gampang diubah — ganti gaya satu situs cukup edit satu file CSS, nggak perlu buka HTML tiap halaman satu-satu.

Lebih cepat — file CSS di-cache browser, jadi saat pindah halaman nggak perlu diunduh ulang.



A.3. Perhatikan aturan `p { color: red; text-align: center; }`. Tunjukkan bagian mana yang disebut selector, properti, nilai, dan blok deklarasi.

Jawaban:

yaitu: `p { color: red; text-align: center; }`

- Selector → `p` (menentukan elemen mana yang di-style)
- Blok deklarasi → `{ color: red; text-align: center; }`
- Di dalam blok deklarasi itu ada dua deklarasi, masing-masing terdiri dari:
 - Properti → `color` dan `text-align`
 - Nilai → `red` dan `center`

A.4. Jelaskan perbedaan selector class dan selector id, lalu sebutkan satu situasi yang tepat untuk masing-masing.

Jawaban:

Selector class (`.nama`) → bisa dipakai berulang kali untuk banyak elemen berbeda. Cocok kalau mau styling sekelompok elemen yang sama,

Selector id (`#nama`) → hanya boleh dipakai sekali per halaman (unik). Cocok untuk elemen yang benar-benar satu-satunya, misal header utama

A.5. Sebuah div diberi `width: 320px`, `padding: 10px`, dan `border: 5px solid gray`. Berapa lebar total elemen tersebut pada standard box model, dan berapa jadinya bila ditambahkan `box-sizing: border-box`?

Jawaban:

Standard box model: `width` cuman menghitung konten, jadi `padding` & `border` ditambahkan di luar.

Lebar total = $320 + (10 \times 2) + (5 \times 2) = 350\text{px}$

Dengan `box-sizing: border-box`: `width` sudah mencakup `padding` & `border`, jadi tidak ditambah lagi.

Lebar total = `320px`



A.6. Jelaskan perbedaan unit em dan rem, lalu sebutkan satu kondisi ketika vh lebih tepat dipakai dibanding px.

Jawaban:

em → relatif terhadap ukuran font elemen induknya (parent). Kalau parent punya font-size berbeda-beda, ukuran em bisa "menumpuk" (compounding) dan jadi susah diprediksi kalau nested.

rem (root em) → relatif terhadap ukuran font elemen root (<html>), jadi nilainya selalu konsisten di mana pun dipakai, nggak peduli nested di parent apa.

A.7. Jelaskan maksud pendekatan mobile-first, dan mengapa breakpoint sebaiknya ditentukan berdasarkan kondisi konten, bukan berdasarkan ukuran perangkat tertentu.

Jawaban:

Pendekatan menulis CSS dengan style dasar untuk layar kecil (mobile) dulu, baru kemudian ditambahkan style untuk layar lebih besa. Jadi arah pengembangannya "dari kecil ke besar", bukan sebaliknya.

Ukuran device itu sangat beragam dan terus berubah (banyak merk, model, resolusi baru bermunculan). Kalau breakpoint dipatok ke device spesifik (misal "iPhone 12 = 390px"), aturan itu cepat usang dan nggak mencakup ke device lain.

nah yg pnting itu kapan tampilan mulai terlihat rusak/tidak nyaman — misal teks jadi terlalu panjang per baris, gambar kepotong, atau layout kaku. Nah, breakpoint sebaiknya dipasang tepat di titik itu, apa pun ukuran device-nya.



B. Identifikasi Selector

Lengkapi kolom sintaks dan situasi pemakaian untuk setiap jenis selector.

Jenis Selector	Sintaks	Kapan dipakai
Universal	*	Untuk menerapkan style ke semua elemen sekaligus, misal reset margin/padding di awal project (* { margin: 0; padding: 0; })
Element / Type	p, div, h1	Untuk menerapkan style ke semua elemen dengan tag yang sama, misal semua paragraf: p { color: gray; }
Class	.nama	untuk elemen yg bisa dipakai berulang di banyak tempat
ID	#nama	untuk elemen yg unik, hanya satu di halaman, misal di header utama
Group	h1, h2, p	untuk menerapkan style yg sama ke beberapa selector berbeda sekaligus
Descendant	div p	Untuk menargetkan elemen yang ada di dalam elemen lain (nested), tanpa peduli seberapa dalam levelnya, misal semua p di dalam div
Child	div > v	Untuk menargetkan elemen yang merupakan anak langsung (satu level di bawah), bukan cucu/lebih dalam



Jenis Selector	Sintaks	Kapan dipakai
Pseudo-class	a:hover	Untuk styling berdasarkan kondisi/state elemen, misal saat di-hover, di-klik, atau elemen pertama: :hover, :first-child
Pseudo-element	p :: first - letter	untuk style bagian tertentu dari suatu elemen, misal huruf pertama atau menyisipkan konten sebelum/sesudah elemen: :: before, :: after



C. Melengkapi Kode

Perhatikan potongan kode berikut. Isi setiap bagian bernomor dengan kode yang benar.

index.html

```
<head>
  <link (1)="stylesheet" (2)="style.css">
</head>
```

style.css

```
* {
  (3): border-box;
}
body {
  font-family: "Poppins", Arial, sans-serif;
  font-size: 1(4);      /* relatif terhadap elemen root */
}
(5)navbar {             /* menyasar elemen dengan id="navbar" */
  display: (6);
  justify-content: space-between;
}
.kartu:(7) {             /* saat kursor berada di atas kartu */
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}
(8) (min-width: 768px) {
  .daftar { grid-template-columns: repeat(3, 1fr); }
}
```

No	Jawaban	Alasan singkat
1	rel	Atribut untuk menyatakan hubungan/jenis file yang dihubungkan, di sini "stylesheet"
2	href	Atribut untuk menunjuk lokasi/path file CSS yang dihubungkan
3	box-sizing	Properti yang mengatur cara perhitungan width/height elemen (termasuk padding & border atau tidak)
4	rem	Unit yang relatif terhadap ukuran font elemen root (<html>), sesuai komentar di kode



No	Jawaban	Alasan singkat
5	#	Selector ID, karena teks komentar bilang "menyasar elemen dengan id=navbar"
6	flex	Supaya <code>justify-content: space-between</code> bisa bekerja, elemen harus jadi flex container
7	hover	Pseudo-class yang aktif "saat kursor berada di atas" elemen
8	@media	Digunakan untuk membuat aturan responsif berdasarkan kondisi ukuran layar (di sini <code>min-width: 768px</code>)



D. Memilih Properti dan Unit

Tentukan properti atau unit CSS yang paling tepat untuk setiap kebutuhan, lalu tulis satu baris contoh kodenya.

Kebutuhan tampilan	Properti / unit	Contoh baris kode
Memberi jarak antara isi kotak dengan garis border	padding	padding: 10px;
Memberi jarak antara satu kartu dengan kartu di sebelahnya	margin atay gap kalau parennta flex/grid	margin-right: 16px;
Membuat section setinggi satu layar penuh	height + unit vh	height : 100vh;
Ukuran huruf yang mengacu pada font-size elemen root	unit rem	font-size : 1,5rem;
Menyembunyikan elemen tanpa menyisakan ruang kosong	display	display : none;
Membuat navigasi tetap terlihat meskipun halaman digulir	position	position: fixed; top : 0;
Menyusun menu navigasi berjajar dengan jarak rata di sepanjang baris	display: flex + justify content	display: flex; justify-content: space-between;
Membagi area menjadi tiga kolom yang sama lebar	display: grid + grid-template-columns	grid-template-columns: repeat(3, 1fr);
Membuat gambar latar memenuhi seluruh area	background-size + background-rep	background-size: cover; background-repeat:



Kebutuhan tampilan	Properti / unit	Contoh baris kode
tanpa diulang	eat	no-repeat;
Mengatur jarak antar baris teks pada paragraf	line-height	line-height: 1.6;



E. Analisis dan Perbaikan Kode

Kode berikut berhasil mengubah tampilan halaman, tetapi ditulis dengan kebiasaan yang buruk.

```
<h1 style="color:#1a1a1a; font-size:34px; text-align:center;">Toko
Kopi</h1>

<style>
  #judul { color: #1a1a1a !important; }
  div div div p { color: #444 !important; font-size: 14px; }
  .kartu { width: 300px; padding: 20px; border: 2px solid #ddd; }
  .kartu { width: 320px; }
  #daftar { display: block; }
  #daftar div { float: left; margin-left: 24px; }
</style>
```

4

E.1. Sebutkan minimal empat masalah pada kode di atas, mulai dari pemakaian inline style, !important, spesifisitas yang berlebihan, sampai aturan yang saling menimpa.

Jawaban:

Inline style di <h1> — style ditulis langsung di atribut style, campur sama struktur HTML. Susah di-maintain karena kalau mau ganti warna/ukuran harus bongkar HTML-nya, bukan cukup edit CSS.

Pemakaian !important berlebihan — dipakai di #judul dan div div div p padahal nggak perlu. !important paksa override semua rule lain, bikin susah di-debug dan menyulitkan override berikutnya kalau butuh perubahan lagi.

Spesifisitas berlebihan — selector div div div p terlalu dalam dan rapuh (bergantung struktur nested yang spesifik banget). Kalau struktur HTML berubah dikit aja, style-nya bisa nggak kena lagi.

Aturan saling menimpa (redundant) — .kartu didefinisikan dua kali dengan width berbeda (300px lalu 320px). Rule kedua otomatis menimpa yang pertama, jadi rule pertama sia-sia ditulis — harusnya cukup satu aturan yang benar dari awal.



E.2. Tulis ulang kode tersebut menjadi CSS eksternal yang rapi. Pakai selector class, hilangkan !important yang tidak perlu, terapkan box-sizing, dan susun daftar kartu memakai Flexbox atau Grid.

Kode perbaikan:

```
<!-- index.html -->
<h1 class="judul">Toko Kopi</h1>

<div class="daftar">
  <div class="kartu">
    <p>Isi kartu...</p>
  </div>
  <div class="kartu">
    <p>Isi kartu...</p>
  </div>
</div>
```



F. Studi Kasus Layout

Sebuah halaman memiliki struktur HTML berikut.

```
<nav class="navbar">
  <span class="logo">KopiKita</span>
  <ul class="menu">
    <li><a href="#">Beranda</a></li>
    <li><a href="#">Menu</a></li>
    <li><a href="#">Kontak</a></li>
  </ul>
</nav>

<main class="daftar">
  <article class="kartu">...</article>    <!-- ada 6 kartu -->
</main>
```

Tulis kode CSS untuk memenuhi ketentuan berikut.

- Navbar memakai Flexbox, logo berada di kiri dan menu di kanan, seluruh isinya rata tengah secara vertikal.
- Item menu berjajar mendatar dengan jarak antar item 16px dan tanpa garis bawah pada tautan.
- Daftar kartu memakai CSS Grid tiga kolom sama lebar dengan gap 20px.
- Setiap kartu memiliki padding, border, dan sudut membulat.
- Pada layar dengan lebar maksimal 480px, daftar kartu berubah menjadi satu kolom dan menu navigasi tersusun ke bawah.

Kode CSS:

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.menu {
  display: flex;
  gap: 16px;
  list-style: none;
}
```



```
.menu a {
  text-decoration: none;
}

.daftar {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.kartu {
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

@media (max-width: 480px) {
  .daftar {
    grid-template-columns: 1fr;
  }

  .navbar {
    flex-direction: column;
  }
}
```



G. Hands On

Bangun halaman "Festival Kuliner Kampus" satu halaman penuh (single page) memakai HTML dan CSS eksternal, lalu isi laporan singkat di bawahnya.

Ketentuan pengerjaan

No	Ketentuan	Selesai
1	Berkas terpisah: index.html dan style.css yang dihubungkan memakai tag <link>. Tidak boleh ada inline style.	☒
2	Bagian hero berisi nama festival, tanggal, dan lokasi, dengan background berwarna atau bergambar serta teks yang tetap terbaca.	☒
3	Navigasi di bagian atas disusun memakai Flexbox: logo di kiri, menu di kanan, dan berubah warna saat kursor berada di atasnya.	☒
4	Daftar minimal enam tenant ditampilkan sebagai kartu memakai CSS Grid, setiap kartu memuat nama tenant, menu andalan, dan harga.	☒
5	Menerapkan satu palet warna yang konsisten dan pemakaian unit rem untuk ukuran huruf.	☒
6	Pada layar sempit, grid tenant berubah menjadi satu kolom dan menu navigasi tersusun ke bawah, memakai media query.	☒

Laporan pengerjaan

G.1. Tempelkan bagian CSS yang mengatur layout, yaitu aturan Flexbox pada navigasi, Grid pada daftar tenant, dan media query yang kamu pakai.

Kode layout:

```
* {
  box-sizing: border-box;
```



```

margin: 0;
padding: 0;
}

body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
  color: #222;
}

/* Navbar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: #222;
}

.logo {
  color: #fff;
  font-weight: bold;
  font-size: 1.2rem;
}

.menu {
  display: flex;
  gap: 1.5rem;
  list-style: none;
}

.menu a {
  color: #eee;
  text-decoration: none;
}

.menu a:hover {
  color: #e0a53b;
}

/* Hero */
.hero {

```



```

height: 60vh;
background: #b23a2e;
color: #fff;
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
text-align: center;
padding: 1rem;
}

.hero h1 {
font-size: 2.5rem;
margin-bottom: 0.5rem;
}

/* Section */
main {
max-width: 60rem;
margin: 0 auto;
padding: 3rem 1.5rem;
}

h2 {
margin-bottom: 1.5rem;
}

/* Daftar tenant (Grid) */
.daftar {
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 1.5rem;
}

.kartu {
border: 1px solid #ddd;
border-radius: 0.5rem;
padding: 1rem;
}

/* Footer */
footer {

```



```
background: #222;
color: #eee;
text-align: center;
padding: 1rem;
}

/* Responsif */
@media (max-width: 480px) {
  .navbar {
    flex-direction: column;
    gap: 0.75rem;
  }

  .daftar {
    grid-template-columns: 1fr;
  }
}
```

G.3. Sebutkan palet warna yang kamu pakai beserta alasan pemilihannya, dan jelaskan satu kendala yang kamu temui saat menyusun layout.

Jawaban:

karna keren dan cocok untuk mahasiswa upi



Selamat Mengerjakan!!

